

Agentic AI with LangGraph



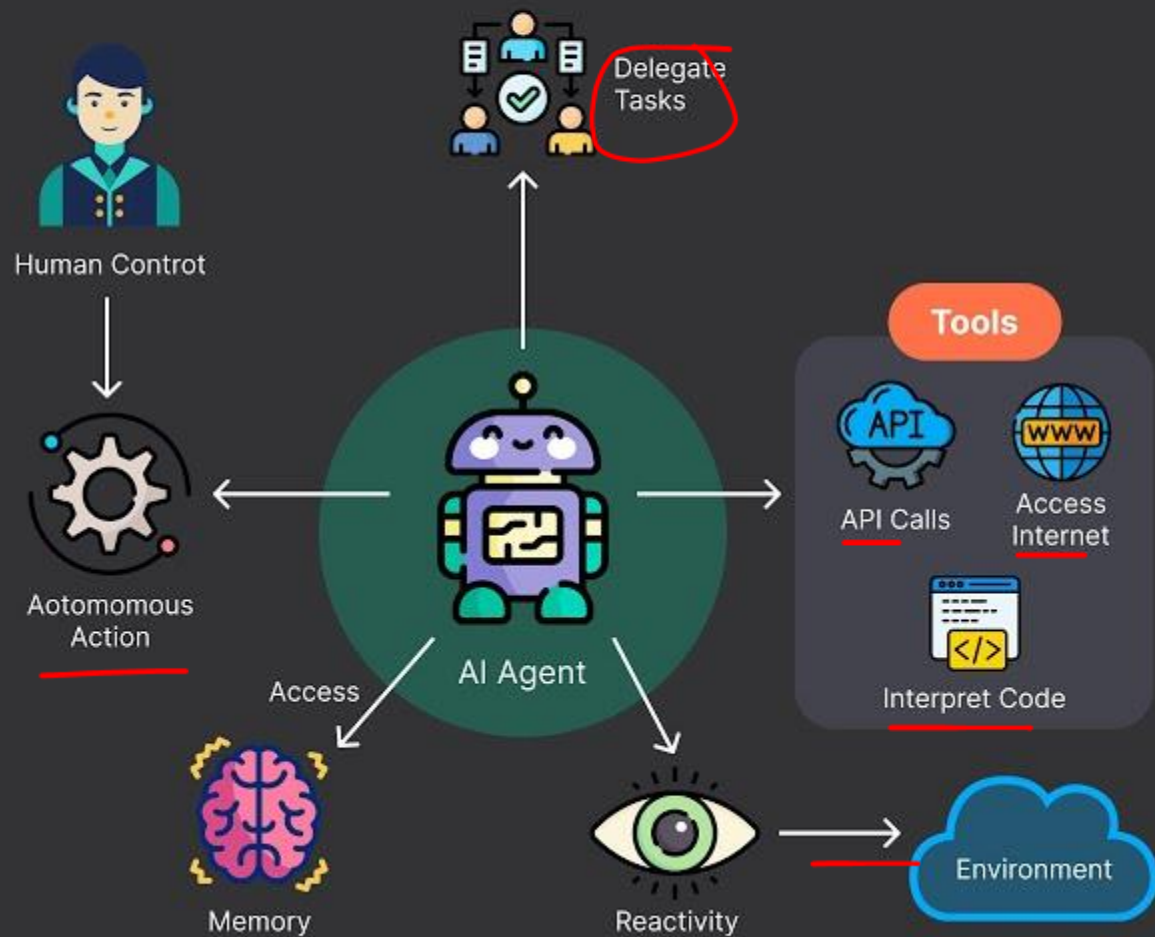
An AI agent is an autonomous system that perceives its environment, makes decisions, plans, and takes actions to achieve specific goals, often using Large Language Models (LLMs) and tools for complex tasks without constant human input, differentiating itself from simple automation by its ability to reason, adapt, and self-improve. They can range from software bots handling customer service to physical robots, performing tasks like research, coding, data analysis, and workflow automation. 

Key Characteristics

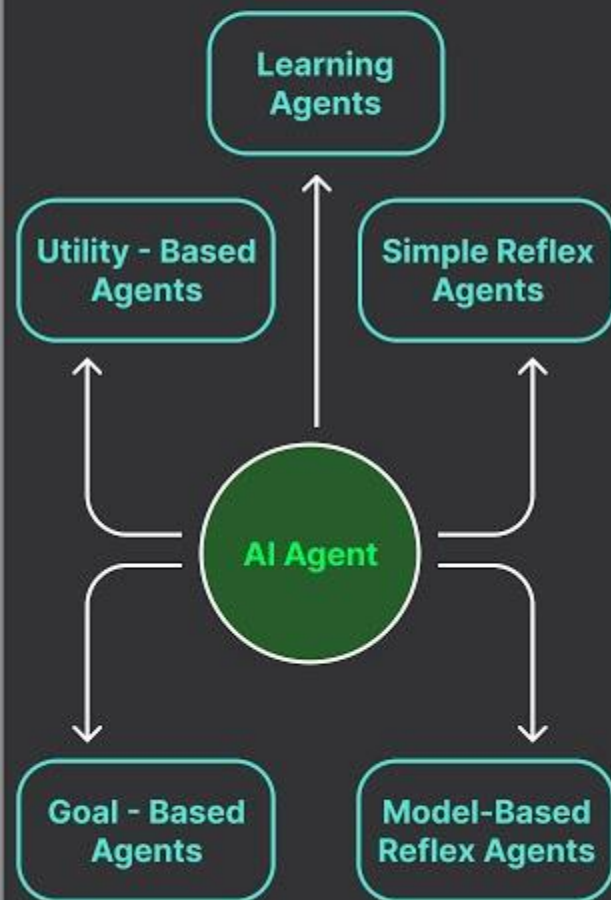
- Autonomy: Acts independently to reach goals, choosing its own steps.
- Perception: Gathers and interprets data from its environment.
- Reasoning & Planning: Uses logic and available info to draw conclusions and strategize.
- Tool Use: Can utilize external tools (internet, code interpreters, APIs) to execute tasks.
- Learning: Improves performance over time through experience and feedback. 

What is an AI Agent?

How AI Agents Work?



Types of AI Agents



AI Agent System Architecture

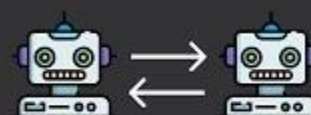
Single Agent

Agents act as personal assistants



Multi-Agent

Agents interact with each other in collaborative ways



Human-Machine

Agents interact with humans to provide assistance



AGENTIC AI vs. AI AGENTS: WHAT'S THE DIFFERENCE?



AGENTIC AI

- Acts on its own
- Complex, multi-step tasks
- Learns and reasons



AI AGENTS

- Needs instructions
- Simple, one task at a time
- Follows predefined rules

OBSERVATION



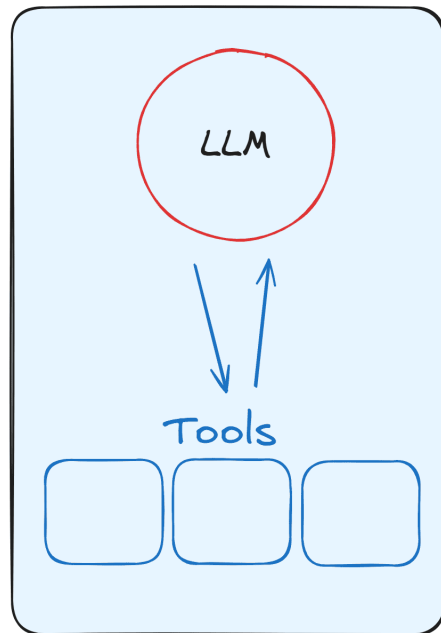
REASONING



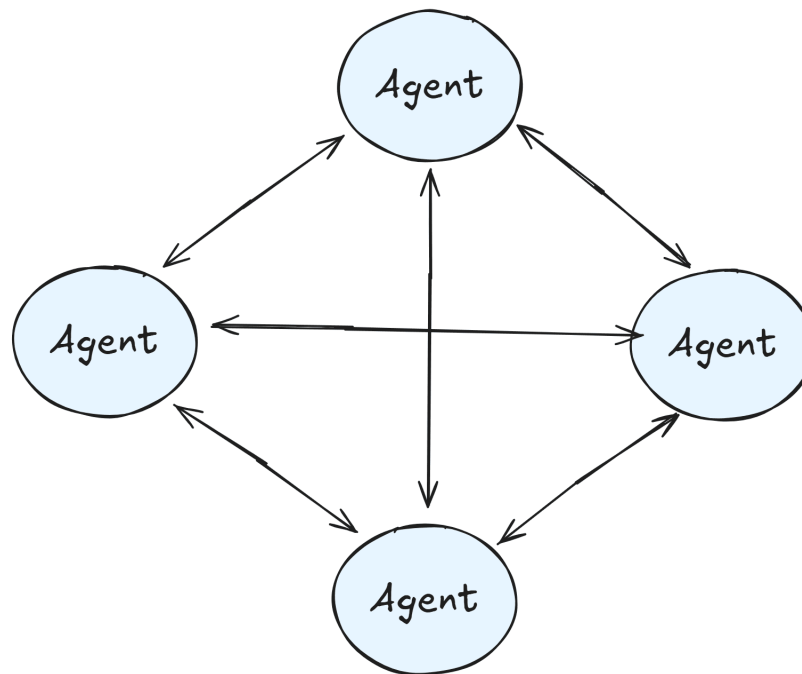
ACTION



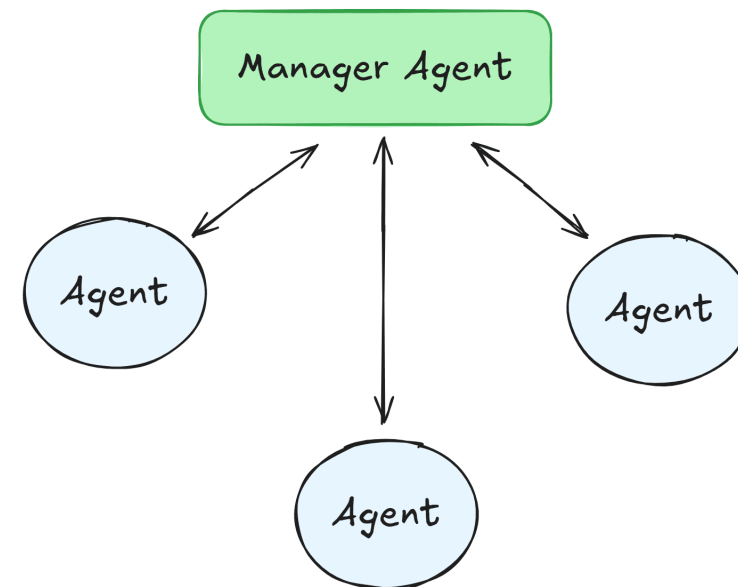
Single Agent Architecture

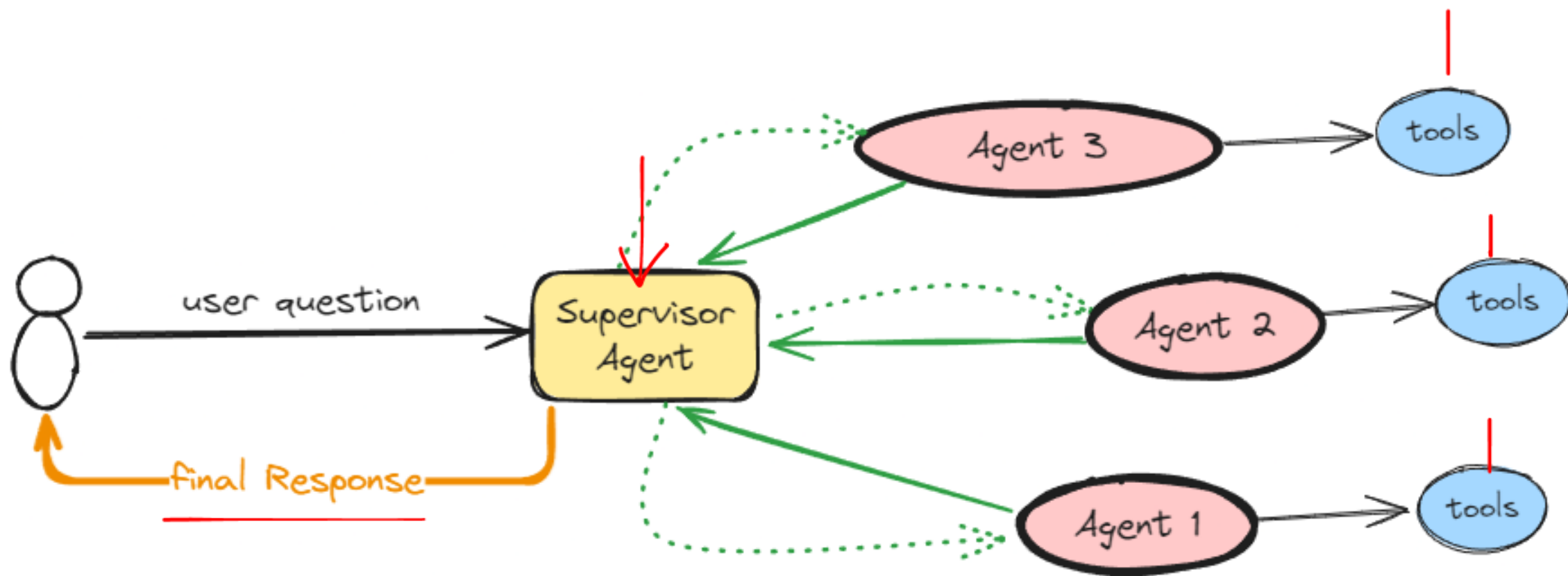


Network Architecture

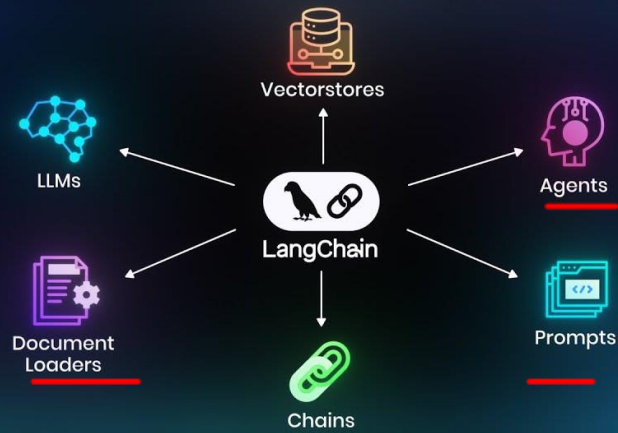


Supervisor Architecture

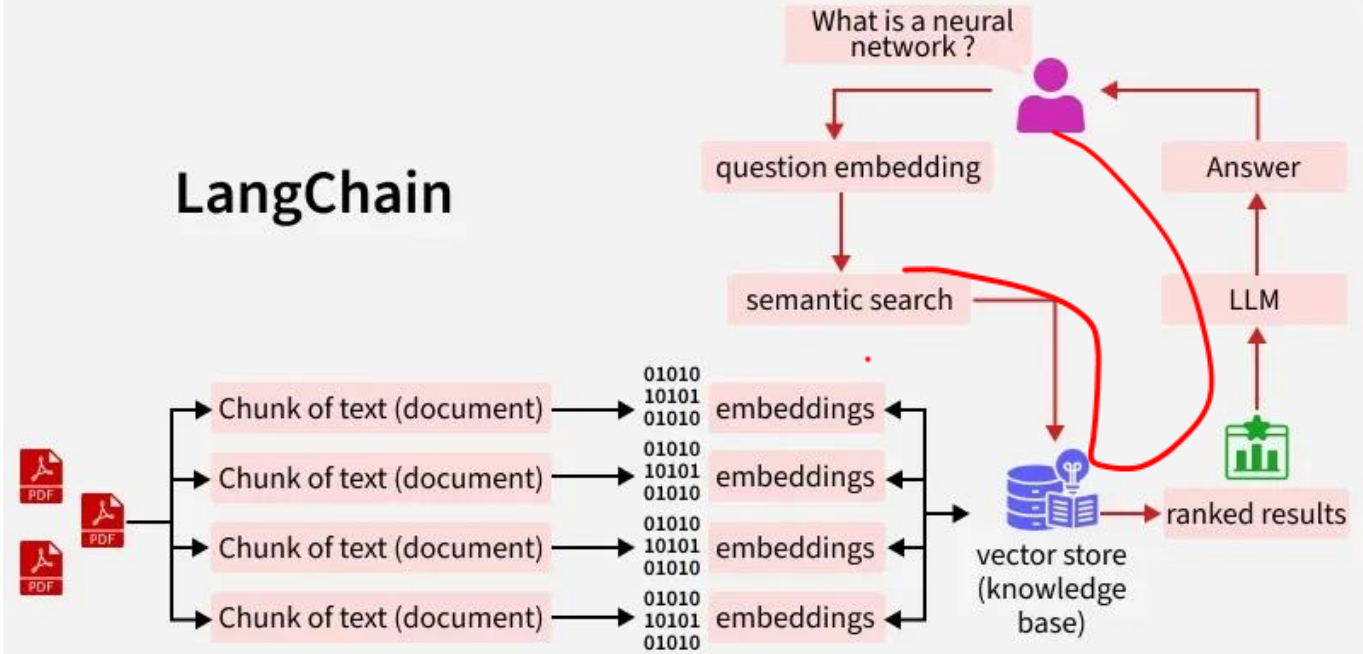




What is LangChain?



LangChain



SIÊU CHI TIẾT VỀ RAG
"BƠM" TRI THỨC CHO LLM

CHATBOT NGÂN HÀNG
TRẢ LỜI SẢN PHẨM DỊCH VỤ
"NGON Ơ"

miai.vn

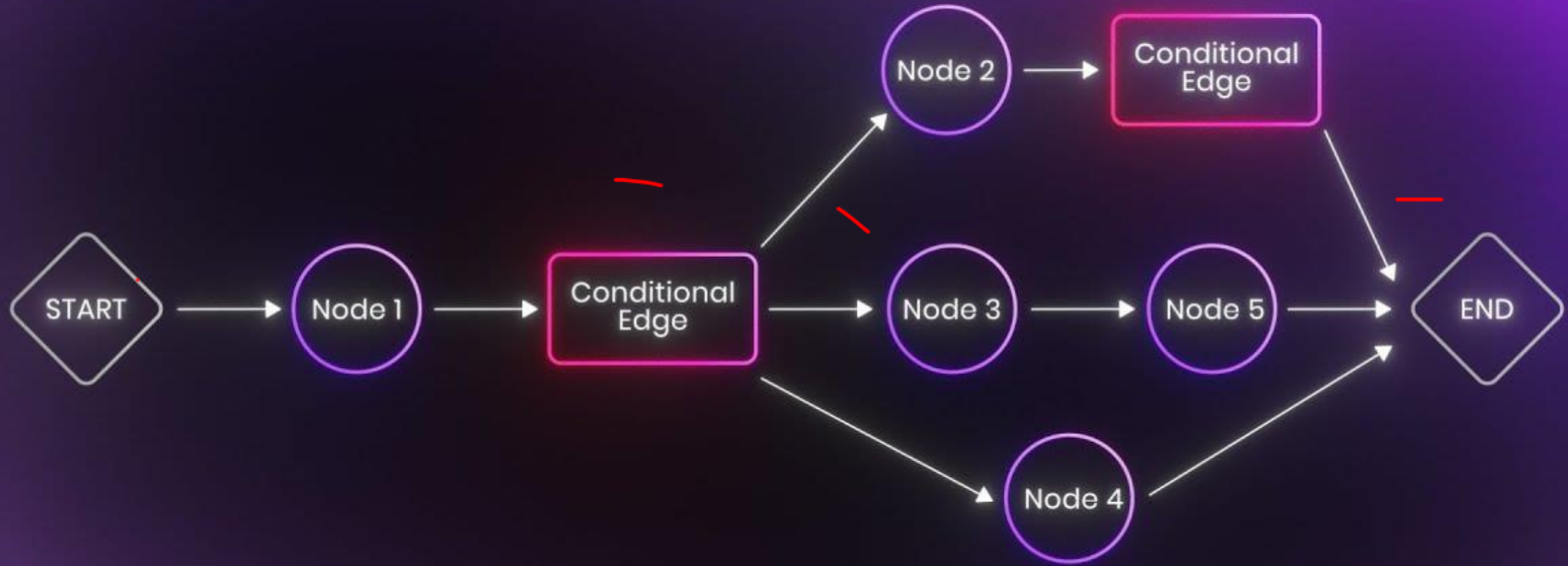
LANGCHAIN - RAG

CLIP #2/2

Hỏi đáp nội dung văn bản
VinaLLama2 Model

VectorDB

What is LangGraph?



LangChain vs LangGraph

Choosing the Right Framework for
Your LLM Project

LangChain

Use When:

- Sequential workflows
- Predictable logic
- Clear pipeline steps

Examples:

- 📄 Document summarizer
- 🔍 Research assistant
- 🗣️ Customer support bot

Architecture:

Simple → Linear pipeline

LangGraph

Use When:

- Dynamic, adaptive logic
- State management needed
- Multi-agent collaboration

Examples:

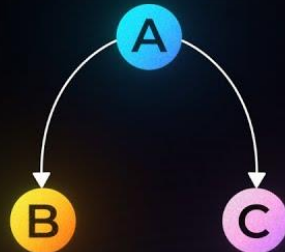
- 📅 Task manager
- 🗣️ Multi-agent chat
- 🛒 Shopping assistant

Architecture:

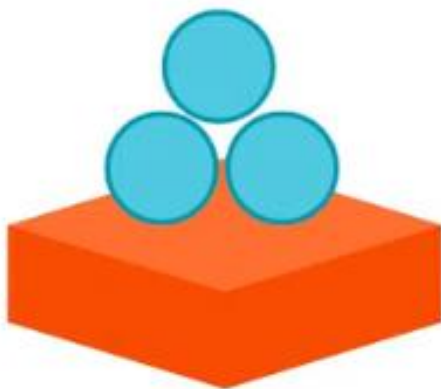
Graph-based →
Revisitable nodes

"Used LangChain for content drafting
and LangGraph for iterative
feedback & edits."

LangChain vs LangGraph



Components of LangGraph



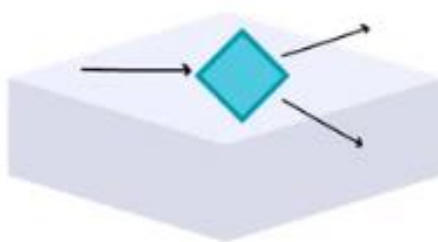
Nodes

nodes represent individual agents, each with a specific role. These agents might be responsible for planning, executing tasks, reviewing outputs, or utilizing specialized tools.



Edges

The edges represent the dynamic connections between agents. These connections adapt based on the context, enabling the system to cycle information through different agents as needed.



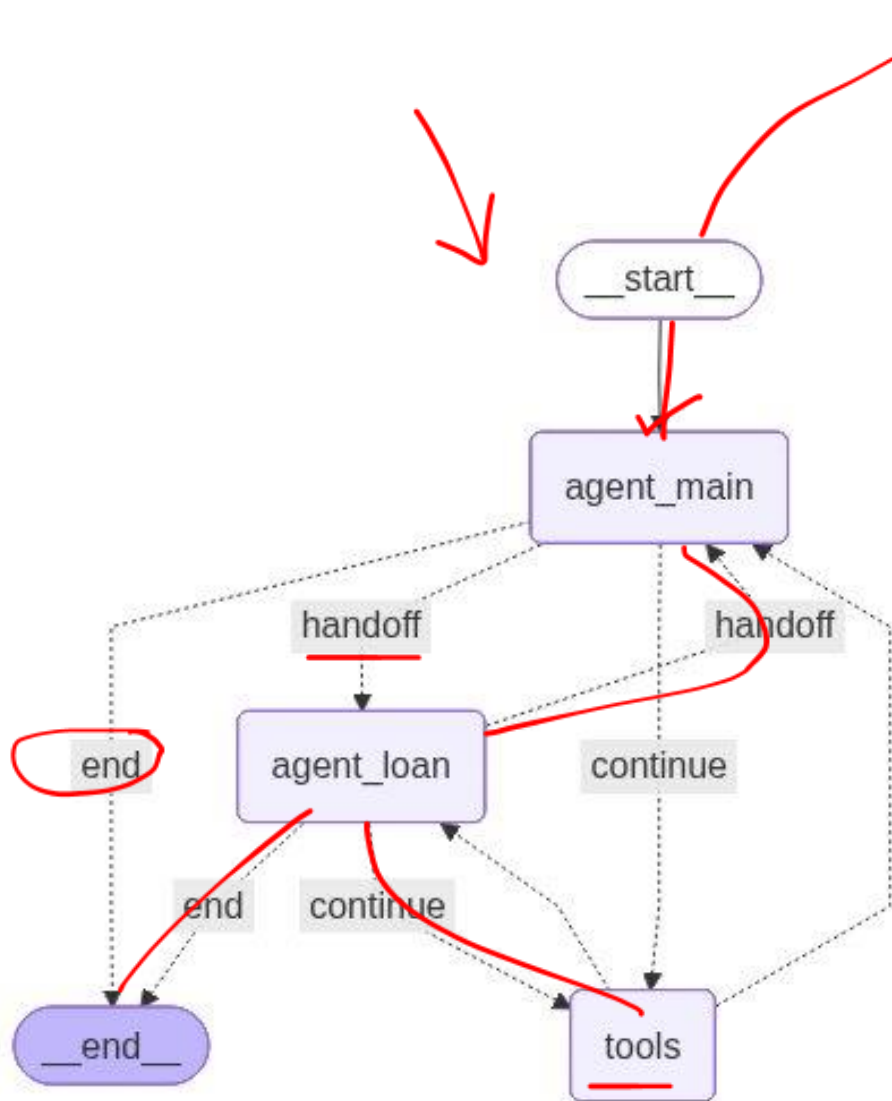
Conditional Edges

Conditional nodes are decision-making points within the graph where the flow of execution is determined based on specific conditions or criteria.



State

State is the information which can be passed between nodes in a whole graph.

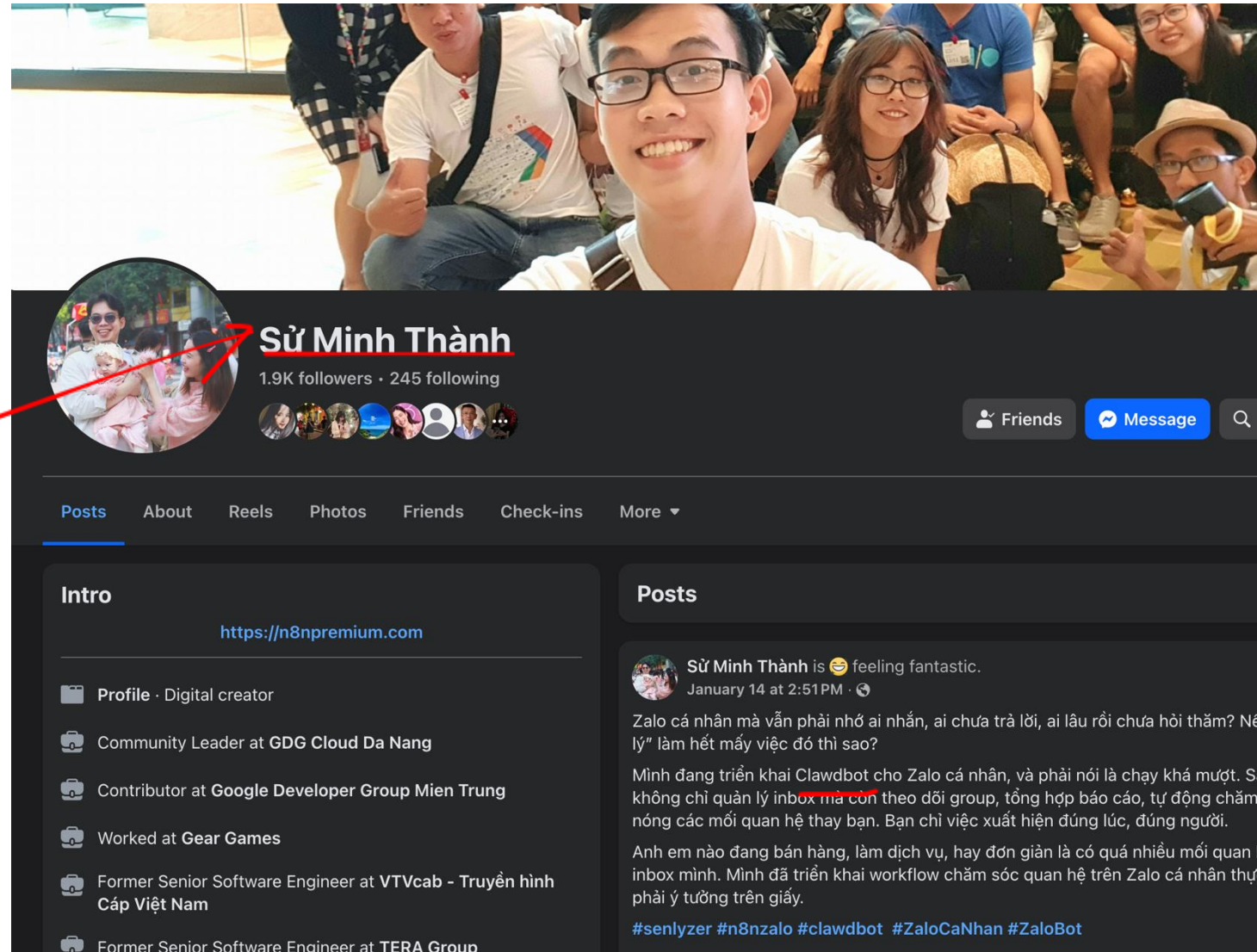


```
agent_state = {  
    "query": "Calculate for me the loan DTI",  
    "last_agent_response": "",  
    "last_agent": "",  
    "tool_observations": [],  
    "num_steps": 0,  
    "customer_no": "0310003896",  
    "user_location": "Hanoi, Vietnam"  
}  
result = agentic_graph_m.invoke(agent_state,  
                                config={"recursion_limit": 10})
```

```
...  
=== 🧠 AGENT_MAIN ===  
HANDOFF: agent_loan  
Route to handoff  
  
=== 🧠 AGENT_LOAN ===  
THOUGHT: The user has requested a calculation of the loan DTI, but no specific details or  
  
ACTION: calculate_dti  
Route to continue  
--- ⚙️ CALLING TOOL OF(agent_loan) ---  
Tool requested: calculate_dti  
Allowed tools = ['calculate_dti']  
--- TOOLCALL: PERFORMING DTI CALCULATION ---  
{'context': 0.6, 'source': 'caculate_dti'}  
  
=== 🧠 AGENT_LOAN ===  
THOUGHT: The DTI has already been calculated and the result is 0.6. I will now provide thi  
  
ANSWER: The debt-to-income ratio (DTI) is 0.6.  
Found ANSWER → ending workflow.
```

Hands on

- Chatbot ngân hàng với Single và MultiAgent.
- Colab
- OpenAI LLM – Thanks to



Hands on

